Lab 2 · wariant b) licznik 0 1 5 2 3 4 6 7

UCiSW 1 · Vivado + AUP-ZU3 · trzybitowy licznik rewersyjny, zadania Z1–Z3, krok po kroku

Zad.	Co trzeba zrobić	Gdzie w tym PDF
Z1	Licznik 3-bitowy z UpNotDown (UPnDN), CE i Rst. W kodzie **3 przerzutniki D z UNISIM** + zminimalizowane równania wzbudzeń. Symulacja behawioralna f_{CLK} = 33 1/3 MHz, pełne cykle w obu kierunkach. Implementacja jako **Block Design**: Clocking Wizard → ClockDivider (1 Hz) → licznik.	teoria + Z1 (część A i B)
Z2	Równania D zastąpione tablicą (`with … select`). Do pokazania **tylko symulacja behawioralna**.	część Z2
Z3	Licznik na Clk_100MHz (bez ClockDiv), kręcony enkoderem: impulsy RotL / RotR dają CE oraz UPnDN.	część Z3

Pliki w folderze `Lab2_b/`

Plik	Do czego	Zadanie
`Licznik_b.vhd`	Licznik (encja **Licznik**): 3× FDRE + równania z map Karnaugha	Z1, Z3
`Licznik_b_tab.vhd`	Ten sam licznik, wzbudzenia z tablicy `with … select`	Z2
`Licznik_tb.vhd`	Testbench: zegar 30 ns, cykl w górę, cykl w dół, CE=0, reset + automatyczne sprawdzanie	Z1, Z2
`Z1.xdc`	Piny dla diagramu Z1: CE ← SW0, UPnDN ← SW1, Rst ← BTN3, Q → LED2…0, Clk_1Hz → LED7	Z1
`Z3.xdc`	Piny dla diagramu Z3: ROT_A, ROT_B (enkoder), Rst, Q	Z3
`ClockDivider.vhd`, `RotaryEnc.vhd`	Moduły od prowadzącego (ePortal, kopie z `pliki/`)	Z1 / Z3
`ZU3.xdc`, `ZU3_Exp.xdc`	Szablony pinów od prowadzącego	—

Plan na zajęcia

Projekt **L2_Z1**: kod + symulacja + diagram + płyta (Z1). Projekt **L2_Z2**: mały, tylko symulacja wersji z tablicą. Projekt **L2_Z3**: kopia L2_Z1 (**Save Project As**), w diagramie podmieniasz dzielnik na enkoder.

Jak czytać instrukcję

Szary prostokąt = przycisk/menu w Vivado · `Ctrl`+`S` = skrót · **PPM** = prawy przycisk myszy · **2×klik** = podwójne kliknięcie · ramka **✔ SPRAWDŹ** = co masz zobaczyć.

Teoria: cykl, tabela przejść, przerzutniki

Cykl wariantu b)

W górę (UPnDN = 1): **0** → **1** → **5** → **2** → **3** → **4** → **6** → **7** → **0**

W dół (UPnDN = 0): **0** → **7** → **6** → **4** → **3** → **2** → **5** → **1** → **0**

W dół idziesz tą samą drogą, tylko w przeciwną stronę.

U	Q teraz	Q następny	$D_2D_1D_0$
1	0 (000)	**1** (001)	`001`
1	1 (001)	**5** (101)	`101`
1	2 (010)	**3** (011)	`011`
1	3 (011)	**4** (100)	`100`
1	4 (100)	**6** (110)	`110`
1	5 (101)	**2** (010)	`010`
1	6 (110)	**7** (111)	`111`
1	7 (111)	**0** (000)	`000`
0	0 (000)	**7** (111)	`111`
0	1 (001)	**0** (000)	`000`
0	2 (010)	**5** (101)	`101`
0	3 (011)	**2** (010)	`010`
0	4 (100)	**3** (011)	`011`
0	5 (101)	**1** (001)	`001`
0	6 (110)	**4** (100)	`100`
0	7 (111)	**6** (110)	`110`

Najważniejsza zasada

Przerzutnik D przy zboczu zegara przepisuje D na Q. Dlatego **D = stan następny**: kolumna D to po prostu kolumna „Q następny” w binarnym.

Dlaczego FDRE, a nie FD

W UltraScale+ (nasz Zynq) podstawowy przerzutnik D z UNISIM to **FDRE**: wejście D, zegar C, **CE** (zezwolenie) i **R** (synchroniczny reset do 0). Stary `FD` w tej rodzinie nie istnieje. CE i Rst obsługuje sam przerzutnik, więc równania D zależą tylko od U, Q_2, Q_1, Q_0 → mapy 4×4.

Co robią CE i Rst

Rst = 1 → przy zboczu Q = 000 (ma pierwszeństwo).
CE = 0 → licznik stoi.
CE = 1 → krok w kierunku UPnDN.

Mapy Karnaugha z grupami

Wiersze U Q_2, kolumny Q_1Q_0 (kod Graya). Górne dwa wiersze (U = 0) to liczenie w dół, dolne (U = 1) w górę. Szara liczba w rogu = obecny stan Q. Kolorowe paski = grupy, kolor = składnik w równaniu. Skrajne kolumny i wiersze są sąsiednie.

D_2 (bit 2 stanu następnego)

U Q_2\Q_1Q_0	00	01	11	10
00	**1** 0	0 1	0 3	**1** 2
01	0 4	0 5	**1** 7	**1** 6
11	**1** 4	0 5	0 7	**1** 6
10	0 0	**1** 1	**1** 3	0 2

Grupy:

- $\overline{U}\,\overline{Q_2}\,\overline{Q_0}$: w dół (U=0): Q = 0, 2
- $\overline{U}\,Q_2\,Q_1$: w dół (U=0): Q = 6, 7
- $U\,\overline{Q_2}\,Q_0$: w górę (U=1): Q = 1, 3
- $U\,Q_2\,\overline{Q_0}$: w górę (U=1): Q = 4, 6

$$D_2 = \overline{U}\,\overline{Q_2}\,\overline{Q_0} + \overline{U}\,Q_2\,Q_1 + U\,\overline{Q_2}\,Q_0 + U\,Q_2\,\overline{Q_0}$$

D_1 (bit 1 stanu następnego)

U Q_2\Q_1Q_0	00	01	11	10
00	**1** 0	0 1	**1** 3	0 2
01	**1** 4	0 5	**1** 7	0 6
11	**1** 4	**1** 5	0 7	**1** 6
10	0 0	0 1	0 3	**1** 2

Grupy:

- $\overline{U}\,\overline{Q_1}\,\overline{Q_0}$: w dół (U=0): Q = 0, 4
- $\overline{U}\,Q_1\,Q_0$: w dół (U=0): Q = 3, 7
- $U\,Q_2\,\overline{Q_1}$: w górę (U=1): Q = 4, 5
- $U\,Q_1\,\overline{Q_0}$: w górę (U=1): Q = 2, 6

$$D_1 = \overline{U}\,\overline{Q_1}\,\overline{Q_0} + \overline{U}\,Q_1\,Q_0 + U\,Q_2\,\overline{Q_1} + U\,Q_1\,\overline{Q_0}$$

D_0 (bit 0 stanu następnego)

U Q_2\Q_1Q_0	00	01	11	10
00	**1** 0	0 1	0 3	**1** 2
01	**1** 4	**1** 5	0 7	0 6
11	0 4	0 5	0 7	**1** 6
10	**1** 0	**1** 1	0 3	**1** 2

Grupy:

- $\overline{U}\,Q_2\,\overline{Q_1}$: w dół (U=0): Q = 4, 5
- $U\,\overline{Q_2}\,\overline{Q_1}$: w górę (U=1): Q = 0, 1
- $U\,Q_1\,\overline{Q_0}$: w górę (U=1): Q = 2, 6
- $\overline{Q_2}\,\overline{Q_0}$: w dół (U=0): Q = 0, 2; w górę (U=1): Q = 0, 2

$$D_0 = \overline{U}\,Q_2\,\overline{Q_1} + U\,\overline{Q_2}\,\overline{Q_1} + U\,Q_1\,\overline{Q_0} + \overline{Q_2}\,\overline{Q_0}$$

Równania (do VHDL)

$$D_2 = \overline{U}\,\overline{Q_2}\,\overline{Q_0} + \overline{U}\,Q_2\,Q_1 + U\,\overline{Q_2}\,Q_0 + U\,Q_2\,\overline{Q_0}$$
$$D_1 = \overline{U}\,\overline{Q_1}\,\overline{Q_0} + \overline{U}\,Q_1\,Q_0 + U\,Q_2\,\overline{Q_1} + U\,Q_1\,\overline{Q_0}$$
$$D_0 = \overline{U}\,Q_2\,\overline{Q_1} + U\,\overline{Q_2}\,\overline{Q_1} + U\,Q_1\,\overline{Q_0} + \overline{Q_2}\,\overline{Q_0}$$

Wszystkie 3 równania sprawdzone skryptem dla 16 kombinacji (U, Q) i w symulacji testbencha.

Ciekawostki do powiedzenia

D_2 dla U = 1: $U\cdot(Q_2 \oplus Q_0)$. D_1 dla U = 0: $\overline{U}\cdot(Q_1 \odot Q_0)$ (XNOR). Każde D zależy od 4 zmiennych → jedna tablica LUT na bit.

Z1 · część A: kod i symulacja

Cel: działający licznik w VHDL i wykres pełnych cykli w obu kierunkach.

1 Nowy projekt

1. Uruchom Vivado → **Create Project** → **Next**.
2. Project name: `L2_Z1`, location: `C:/AMD_Proj`, ☑ Create project subdirectory → **Next**. Bez polskich znaków i spacji!
3. ◉ RTL Project, ☑ Do not specify sources at this time → **Next**.
4. Zakładka **Boards** → szukaj **AUP** → **Zynq UltraScale+ AUP-ZU3 8GB** → **Next** → **Finish**.

✔ SPRAWDŹ Project Summary: Target language = VHDL.

2 Pliki: licznik i dzielnik

1. **Add Sources** (Alt+A) → ◉ Add or create design sources → **Next**.
2. **Add Files** → `Licznik_b.vhd` i `ClockDivider.vhd` → ☑ Copy sources into project → **Finish**.

Kod licznika (z pliku `Licznik_b.vhd`):

```
-- Lab 2 Z1 b) Trzybitowy licznik rewersyjny, cykl w gore: 0 1 5 2 3 4 6 7
-- (w dol odwrotnie: 0 7 6 4 3 2 5 1). 3 przerzutniki FDRE z UNISIM
-- (D z CE i synchronicznym R) + zminimalizowane rownania wzbudzen D.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library UNISIM;
use UNISIM.VComponents.all;

entity Licznik is
    Port ( Clk   : in  STD_LOGIC;
           CE    : in  STD_LOGIC;
           Rst   : in  STD_LOGIC;
           UPnDN : in  STD_LOGIC;
           Q     : out STD_LOGIC_VECTOR (2 downto 0));
end Licznik;

architecture Behavioral of Licznik is
    signal Qi : STD_LOGIC_VECTOR (2 downto 0);  -- wyjscia przerzutnikow
    signal D  : STD_LOGIC_VECTOR (2 downto 0);  -- wzbudzenia
    signal U  : STD_LOGIC;
begin
    U <= UPnDN;

    -- Rownania z map Karnaugha (U = UPnDN)
    D(2) <= (not U and not Qi(2) and not Qi(0)) or (not U and Qi(2) and Qi(1))
         or (U and not Qi(2) and Qi(0)) or (U and Qi(2) and not Qi(0));
    D(1) <= (not U and not Qi(1) and not Qi(0)) or (not U and Qi(1) and Qi(0))
         or (U and Qi(2) and not Qi(1)) or (U and Qi(1) and not Qi(0));
    D(0) <= (not U and Qi(2) and not Qi(1)) or (U and not Qi(2) and not Qi(1))
         or (U and Qi(1) and not Qi(0)) or (not Qi(2) and not Qi(0));

    -- Trzy przerzutniki typu D z biblioteki UNISIM
    FF2: FDRE generic map (INIT => '0')
         port map (C => Clk, CE => CE, R => Rst, D => D(2), Q => Qi(2));
    FF1: FDRE generic map (INIT => '0')
         port map (C => Clk, CE => CE, R => Rst, D => D(1), Q => Qi(1));
    FF0: FDRE generic map (INIT => '0')
         port map (C => Clk, CE => CE, R => Rst, D => D(0), Q => Qi(0));

    Q <= Qi;
end Behavioral;
```

✔ SPRAWDŹ Design Sources: **Licznik** i **ClockDivider**, bez czerwonych kresek. Pod Licznik widać FF2, FF1, FF0 (FDRE).

3 Testbench

1. **Add Sources** → ◉ Add or create simulation sources → **Add Files** → `Licznik_tb.vhd` → ☑ Copy → **Finish**.
2. Simulation Sources → sim_1: **Licznik_tb** ma być pogrubiony (top). Jeśli nie: PPM → **Set as Top**.

```
-- Testbench Lab 2 b): Clk 33 1/3 MHz (okres 30 ns). Reset, pelny cykl w gore,
-- pelny cykl w dol, CE = 0 (stoi), znow w gore, reset w trakcie liczenia.
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Licznik_tb is
end Licznik_tb;

architecture Behavioral of Licznik_tb is
    component Licznik
        Port ( Clk   : in  STD_LOGIC;
               CE    : in  STD_LOGIC;
               Rst   : in  STD_LOGIC;
               UPnDN : in  STD_LOGIC;
               Q     : out STD_LOGIC_VECTOR (2 downto 0));
    end component;

    signal Clk   : STD_LOGIC := '0';
    signal CE    : STD_LOGIC;
    signal Rst   : STD_LOGIC;
    signal UPnDN : STD_LOGIC;
    signal Q     : STD_LOGIC_VECTOR (2 downto 0);
begin
    UUT: Licznik port map ( Clk => Clk, CE => CE, Rst => Rst, UPnDN => UPnDN, Q => Q );

    -- Zegar 33 1/3 MHz: zbocza narastajace w 15, 45, 75, ... ns
    Clk <= not Clk after 15 ns;

    -- Wymuszenia zmieniaja sie miedzy zboczami narastajacymi (co 30 ns)
    Rst   <= '1', '0' after 60 ns, '1' after 690 ns, '0' after 720 ns;
    CE    <= '1', '0' after 540 ns, '1' after 600 ns;
    UPnDN <= '1', '0' after 300 ns, '1' after 540 ns;
    -- 60..300 ns:  8 taktow w gore  0 1 5 2 3 4 6 7 0
    -- 300..540 ns: 8 taktow w dol   0 7 6 4 3 2 5 1 0
    -- 540..600 ns: CE = 0, licznik stoi
    -- 600..690 ns: w gore 1 5 2, potem reset -> 0

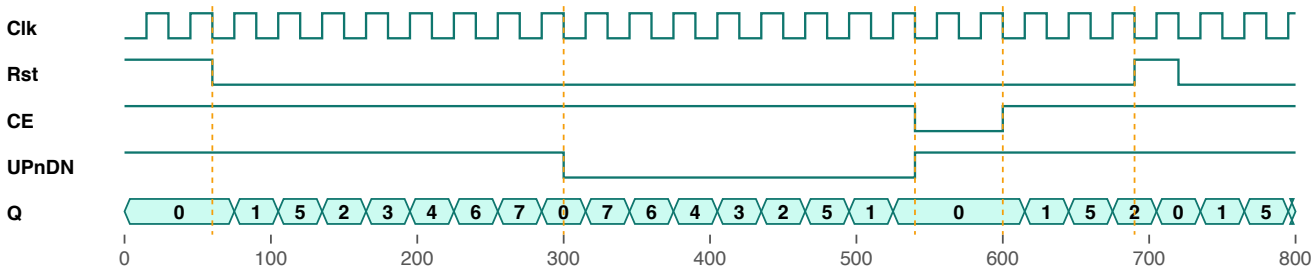
    -- Samosprawdzenie (opcjonalne): model licznika liczony rownolegle
    Check: process
        type Tab is array (0 to 7) of integer;
        constant NAST_UP : Tab := (1, 5, 3, 4, 6, 2, 7, 0);
        constant NAST_DN : Tab := (7, 0, 5, 2, 3, 1, 4, 6);
        variable Ocz : integer := 0;
    begin
        while now < 800 ns loop
            wait until rising_edge(Clk);
            if Rst = '1' then
                Ocz := 0;
            elsif CE = '1' then
                if UPnDN = '1' then Ocz := NAST_UP(Ocz);
                else                Ocz := NAST_DN(Ocz);
                end if;
            end if;
            wait for 10 ns;
            assert to_integer(unsigned(Q)) = Ocz
                report "Blad: Q = " & integer'image(to_integer(unsigned(Q)))
                    & ", oczekiwano " & integer'image(Ocz) severity error;
        end loop;
        report "Koniec testu";
        wait;
    end process;
end Behavioral;
```

Proces `Check` jest opcjonalny: liczy licznik „na boku” i porównuje z Q. Polecenie wymaga zegara (fala okresowa) i zmiany UPnDN.

> **Skąd 30 ns**
>
> f = 33 1/3 MHz → T = 1 / 33,33 MHz = 30 ns → zegar zmienia się co 15 ns: `Clk <= not Clk after 15 ns;`

4 Symulacja behawioralna

1. Flow Navigator → SIMULATION → **Run Simulation** → **Run Behavioral Simulation**.
2. Test trwa ~800 ns, a domyślnie symulacja idzie 1000 ns, więc nic nie dopisujesz.
3. **Zoom Fit** (⛶). PPM na **Q** → **Radix** → **Unsigned Decimal**.



Co masz zobaczyć (narysowane z modelu, zgodne z symulacją). Pomarańczowe linie: start, zmiana kierunku, CE = 0, znów w górę, reset.

✔ SPRAWDŹ Q: 0 1 5 2 3 4 6 7 0 (60–300 ns), potem 7 6 4 3 2 5 1 0 (300–540 ns), stoi przy CE = 0, 1 5 2, reset → 0. Tcl Console: „Koniec testu”, brak „Blad”.

⚠ **Pokaż prowadzącemu**: mapy Karnaugha + ten wykres.

Z1 · część B: diagram blokowy i płyta

Ten sam projekt L2_Z1. Zegar 100 MHz → dzielnik 1 Hz → licznik. Wynik na diodach.

Sygnał	Skąd / dokąd	Na płycie
Clk_100MHz	Clocking Wizard (Board → Differential Clock)	generator 100 MHz
Clk_1Hz	ClockDivider Clk_Out → Licznik Clk oraz port **Clk_1Hz**	LD7 (miga)
CE	port **CE** → Licznik CE	SW0
UPnDN	port **UPnDN** → Licznik UPnDN	SW1
Rst	port **Rst** → Licznik Rst	BTN3 (lewy dolny przycisk)
Q[2:0]	Licznik Q → port **Q[2:0]**	LD2 LD1 LD0

1 XDC

1. **Add Sources** → ◉ Add or create constraints → **Add Files** → `Z1.xdc` → ☑ Copy → **Finish**.

```
## Lab 2 Z1 (diagram blokowy): nazwy portow jak w diagramie.
## Zegara CLK_100MHz_* tu NIE opisujemy -- robi to Clocking Wizard.

## CE <- SW0, UPnDN <- SW1
set_property PACKAGE_PIN AB1 [get_ports {CE}]
set_property PACKAGE_PIN AF1 [get_ports {UPnDN}]
set_property IOSTANDARD LVCMOS12 [get_ports {CE UPnDN}]

## Rst <- PB3
set_property PACKAGE_PIN AC6 [get_ports {Rst}]
set_property IOSTANDARD LVCMOS12 [get_ports {Rst}]

## Q(2:0) -> LED2..LED0
set_property PACKAGE_PIN AF5 [get_ports {Q[0]}]
set_property PACKAGE_PIN AE7 [get_ports {Q[1]}]
set_property PACKAGE_PIN AH2 [get_ports {Q[2]}]
set_property IOSTANDARD LVCMOS12 [get_ports Q*]

## Clk_1Hz -> LED7
set_property PACKAGE_PIN AF2 [get_ports {Clk_1Hz}]
set_property IOSTANDARD LVCMOS12 [get_ports {Clk_1Hz}]
```

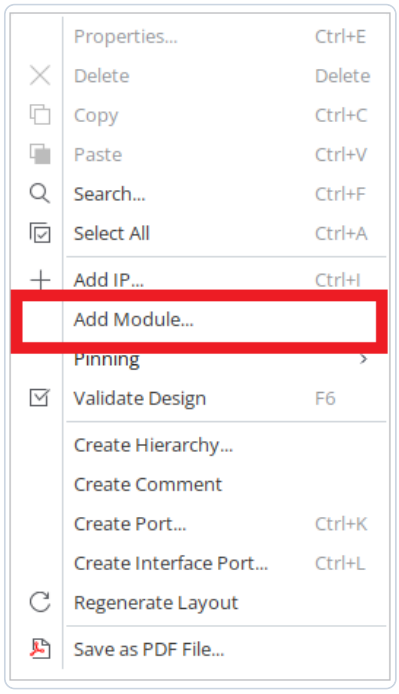
> ⚠ Linii zegara CLK_100MHz_* **nie ma**: robi to Clocking Wizard.

2 Pusty diagram

Flow Navigator → IP INTEGRATOR → **Create Block Design** → nazwa `L2_BD` → **OK**.

3 Klocki Licznik i ClockDivider

PPM na planszy → **Add Module...** → **Licznik** → **OK**. Powtórz dla **ClockDivider**.

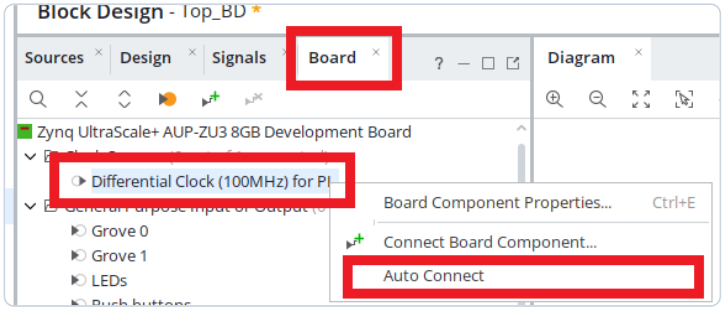


PPM na planszy → Add Module

2×klik na ClockDivider_0: IN_FREQ_HZ = `100E6`, OUT_FREQ_HZ = `1` (to są wartości domyślne, tylko sprawdź).

4 Zegar (Clocking Wizard)

1. Lewy panel → zakładka **Board** → **Differential Clock (100MHz) for PL** → PPM → **Auto Connect**.
2. 2×klik na **clk_wiz_0** → Output Clocks → na dole odznacz □ reset i □ locked → **OK**.

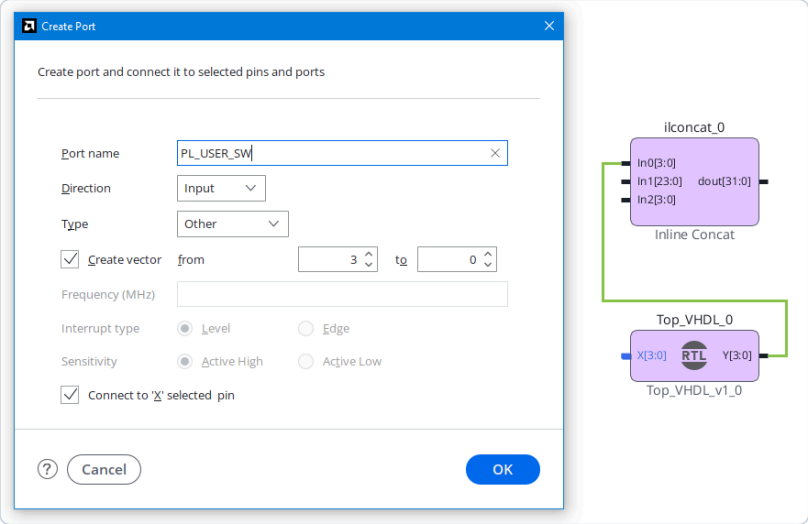


Board → Differential Clock → Auto Connect

5 Porty (Create Port)

PPM na nóżce → **Create Port...** (Ctrl+K), ☑ Connect to selected pin, Type: **Other**:

Nóżka	Port name	Direction	Vector
Licznik_0 CE	CE	Input	—
Licznik_0 UPnDN	UPnDN	Input	—
Licznik_0 Rst	Rst	Input	—
Licznik_0 Q[2:0]	Q	Output	☑ 2 to 0
ClockDivider_0 Clk_Out	Clk_1Hz	Output	—



Przykład okna Create Port (z instrukcji prowadzącego)

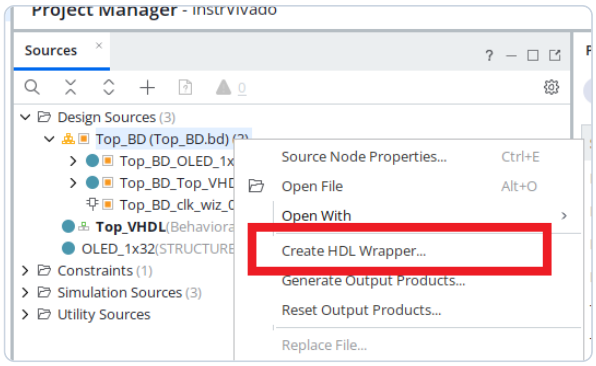
> ⚠ Nazwy muszą być takie jak w `Z1.xdc`. Jeśli użyjesz Make External, dostaniesz `CE_0` itd.: zmień nazwę w External Port Properties.

6 Połączenia (przeciągnij myszką)

#	Od	Do
1	clk_wiz_0 clk_out1	ClockDivider_0 Clk_In
2	ClockDivider_0 Clk_Out	Licznik_0 Clk (Clk_Out ma już port Clk_1Hz, to drugie połączenie)

7 Walidacja, wrapper

1. **Regenerate Layout** (⟳) → **Validate Design** (F6) → Ctrl+S.
2. Sources → PPM na L2_BD (.bd) → **Create HDL Wrapper...** → ◉ Let Vivado manage wrapper and auto-update → **OK**.



Create HDL Wrapper

✔ SPRAWDŹ Validate: „successful". Design Sources: **L2_BD_wrapper** pogrubiony (top). Jeśli top to Licznik: PPM na wrapper → Set as Top.

8 Bitstream, wgranie, test

1. **Generate Bitstream** → **Yes** → **OK**. Czekaj na **write_bitstream Complete**.
2. **Open Hardware Manager** → **Open target** → **Auto Connect** → **Program device** → `L2_BD_wrapper.bit` → **Program**.

SW1 (UPnDN)	SW0 (CE)	BTN3	LD2 LD1 LD0 (co sekundę)
1	1	—	0 1 5 2 3 4 6 7 0 … (w górę)
0	1	—	0 7 6 4 3 2 5 1 0 … (w dół)
x	0	—	stoi
x	x	wciśnięty	000

✔ SPRAWDŹ LD7 miga 1 Hz (0,5 s świeci, 0,5 s nie). Diody LD2…LD0 pokazują Q binarnie (świeci = 1), np. 5 = ●○●.

Z2 · tablica zamiast równań

Wzbudzenia D z instrukcji `with … select`. Do pokazania tylko symulacja behawioralna.

Idea

Zamiast równań wpisujesz tabelę przejść wprost: sklejasz `UPnDN & Q` w 4-bitowy wektor `Sel` i dla każdej z 16 wartości podajesz D. Przerzutniki FDRE zostają te same. Synteza i tak zrobi z tego te same LUT-y.

1 Mały projekt L2_Z2

1. Nowy projekt jak w Z1 (krok 1), nazwa `L2_Z2`.
2. **Add Sources** → design sources → **Add Files** → `Licznik_b_tab.vhd` → ☑ Copy → **Finish**.
3. **Add Sources** → simulation sources → **Add Files** → `Licznik_tb.vhd` (ten sam testbench!) → ☑ Copy → **Finish**.

⚠ Obie wersje mają encję **Licznik**. Dlatego osobny projekt: w jednym projekcie nie mogą być dwa pliki z tą samą encją.

2 Kod

```
-- Lab 2 Z2 b) Ten sam licznik (cykl w gore: 0 1 5 2 3 4 6 7), ale wzbudzenia D
-- z tablicy (with ... select) zamiast rownan. Przerzutniki bez zmian (FDRE).
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
library UNISIM;
use UNISIM.VComponents.all;

entity Licznik is
    Port ( Clk   : in  STD_LOGIC;
           CE    : in  STD_LOGIC;
           Rst   : in  STD_LOGIC;
           UPnDN : in  STD_LOGIC;
           Q     : out STD_LOGIC_VECTOR (2 downto 0));
end Licznik;

architecture Behavioral of Licznik is
    signal Qi  : STD_LOGIC_VECTOR (2 downto 0);
    signal D   : STD_LOGIC_VECTOR (2 downto 0);
    signal Sel : STD_LOGIC_VECTOR (3 downto 0);  -- UPnDN & Q
begin
    Sel <= UPnDN & Qi;

    with Sel select
        D <= "111" when "0000",   -- w dol: 0 -> 7
             "000" when "0001",   --        1 -> 0
             "101" when "0010",   --        2 -> 5
             "010" when "0011",   --        3 -> 2
             "011" when "0100",   --        4 -> 3
             "001" when "0101",   --        5 -> 1
             "100" when "0110",   --        6 -> 4
             "110" when "0111",   --        7 -> 6
             "001" when "1000",   -- w gore: 0 -> 1
             "101" when "1001",   --         1 -> 5
             "011" when "1010",   --         2 -> 3
             "100" when "1011",   --         3 -> 4
             "110" when "1100",   --         4 -> 6
             "010" when "1101",   --         5 -> 2
             "111" when "1110",   --         6 -> 7
             "000" when "1111",   --         7 -> 0
             "000" when others;

    FF2: FDRE generic map (INIT => '0')
         port map (C => Clk, CE => CE, R => Rst, D => D(2), Q => Qi(2));
    FF1: FDRE generic map (INIT => '0')
         port map (C => Clk, CE => CE, R => Rst, D => D(1), Q => Qi(1));
    FF0: FDRE generic map (INIT => '0')
         port map (C => Clk, CE => CE, R => Rst, D => D(0), Q => Qi(0));

    Q <= Qi;
end Behavioral;
```

`Sel` to sygnał pośredni, bo w VHDL-93 nie wolno pisać `with UPnDN & Qi select`. `when others` jest wymagane (std_logic ma też wartości U, X, Z…).

3 Symulacja

Run Simulation → **Run Behavioral Simulation** → Zoom Fit → Q jako Unsigned Decimal.

✔ **SPRAWDŹ** Wykres **identyczny** jak w Z1 (ta sama sekwencja). „Koniec testu”, brak „Blad”.

⚠ **Pokaż prowadzącemu**: kod z `with … select` + wykres.

Z3 · licznik kręcony enkoderem

Zegar 100 MHz bez dzielnika. Obrót w prawo = krok w górę, w lewo = krok w dół.

Jak to działa

Moduł **RotaryEnc** (ePortal) daje na wyjściu **RotR** lub **RotL** impuls trwający **1 takt** zegara przy każdym „kliknięciu” pokrętła.

• **CE = RotL or RotR**: licznik robi krok tylko w takcie impulsu.

• **UPnDN = RotR**: przy impulsie RotR jest 1 (w górę), przy RotL jest 0 (w dół). Poza impulsem UPnDN nie ma znaczenia, bo CE = 0.

Sygnał	Skąd / dokąd	Na płycie
ROT_A, ROT_B	porty → RotaryEnc_0	enkoder na płycie rozszerzającej
Clk	clk_wiz_0 clk_out1 → RotaryEnc_0 Clk i Licznik_0 Clk	100 MHz
CE	Utility Vector Logic (OR) Res → Licznik CE	—
UPnDN	RotaryEnc_0 RotR → Licznik UPnDN	—
Rst	port Rst	BTN3
Q[2:0]	port Q	LD2…LD0

1 Kopia projektu

1. W projekcie L2_Z1: **File** → **Project** → **Save As...** → nazwa `L2_Z3` → **OK**. Teraz pracujesz na kopii, L2_Z1 zostaje do pokazania.
2. **Add Sources** → design sources → **Add Files** → `RotaryEnc.vhd` → ☑ Copy → **Finish**.
3. Constraints: PPM na **Z1.xdc** → **Remove File from Project**. **Add Sources** → constraints → `Z3.xdc` → ☑ Copy → **Finish**.

```
## Lab 2 Z3 (diagram blokowy): licznik na 100 MHz, CE i UPnDN z enkodera.
## Zegara CLK_100MHz_* tu NIE opisujemy -- robi to Clocking Wizard.

## Enkoder obrotowy (plyta rozszerzajaca, linie z ZU3_Exp.xdc)
set_property PACKAGE_PIN AA8 [get_ports {ROT_A}]
set_property PACKAGE_PIN Y9  [get_ports {ROT_B}]
set_property IOSTANDARD LVCMOS33 [get_ports ROT_*]

## Rst <- PB3
set_property PACKAGE_PIN AC6 [get_ports {Rst}]
set_property IOSTANDARD LVCMOS12 [get_ports {Rst}]

## Q(2:0) -> LED2..LED0
set_property PACKAGE_PIN AF5 [get_ports {Q[0]}]
set_property PACKAGE_PIN AE7 [get_ports {Q[1]}]
set_property PACKAGE_PIN AH2 [get_ports {Q[2]}]
set_property IOSTANDARD LVCMOS12 [get_ports Q*]
```

2 Usuń z diagramu

Open Block Design. Zaznacz i usuń (`Delete`): **ClockDivider_0**, porty **Clk_1Hz**, **CE**, **UPnDN**.

3 Dodaj klocki

1. PPM na planszy → **Add Module...** → **RotaryEnc** → **OK**.
2. **+ Add IP** (`Ctrl`+`I`) → wpisz **vector logic** → **Utility Vector Logic**. 2×klik: C_SIZE = `1`, C_OPERATION = **or** → **OK**.

Nazwa może być „Inline Utility Vector Logic” (zależy od wersji Vivado). Wejścia Op1[0:0], Op2[0:0], wyjście Res[0:0]; 1-bitowy wektor łączy się z sygnałem std_logic bez problemu.

4 Porty enkodera

PPM na nóżce ROT_A klocka RotaryEnc_0 → **Create Port...**: **ROT_A**, Input. To samo dla **ROT_B**.

5 Połączenia

#	Od	Do
1	clk_wiz_0 clk_out1	Licznik_0 Clk (po usunięciu dzielnika ta nóżka jest wolna)
2	clk_wiz_0 clk_out1	RotaryEnc_0 Clk
3	RotaryEnc_0 RotR	util_vector_logic_0 Op1
4	RotaryEnc_0 RotL	util_vector_logic_0 Op2
5	util_vector_logic_0 Res	Licznik_0 CE
6	RotaryEnc_0 RotR	Licznik_0 UPnDN

6 Walidacja, bitstream, test

1. **Regenerate Layout** → **Validate Design** (F6) → Ctrl+S. Wrapper zaktualizuje się sam.
2. **Generate Bitstream** → **Program device** (`L2_BD_wrapper.bit` w folderze projektu **L2_Z3**).

Ruch	LD2 LD1 LD0 (kolejne kliknięcia)
w prawo	0 → 1 → 5 → 2 → 3 → 4 → 6 → 7 → 0
w lewo	0 → 7 → 6 → 4 → 3 → 2 → 5 → 1 → 0
BTN3	000

✔ **SPRAWDŹ** Jedno kliknięcie pokrętła = jeden krok licznika. Licznik stoi, gdy nie kręcisz.

Coś nie działa

Objaw	Co zrobić
Synteza/symulacja: nie zna FDRE	Brak `library UNISIM; use UNISIM.VComponents.all;` na górze pliku.
Q = „U” (czerwone) w symulacji	Hierarchia: Licznik_tb → UUT : Licznik. Set as Top na Licznik_tb. Sprawdź Clk w testbenchu: `:= '0'`.
Wykres stoi w 0	Rst w testbenchu zostało w 1 albo CE = 0. Porównaj z wykresem z Z1.
Kierunek odwrotny (w górę idzie 0 7 6 …)	Zamienione znaczenie UPnDN: 1 = w górę. Sprawdź SW1 / połączenie RotR → UPnDN.
Inna kolejność niż 0 1 5 2 3 4 6 7	Błąd w równaniu D. Porównaj kod z mapami albo uruchom wersję z Z2: jeśli ona działa, błąd jest w równaniach.
Validate: Critical Warning **BD 41-1348** „Reset pin /Licznik_0/Rst … asynchronous reset source /Rst”	**Nic złego, kliknij OK.** Vivado radzi dodać Processor System Reset (synchronizacja resetu). Przy 1 Hz i resecie z przycisku nie ma to znaczenia. Kółko przy nóżce Rst to tylko symbol resetu.
Critical Warning „No ports matched …”	Nazwa w XDC ≠ nazwa portu (np. `CE_0`). Zmień nazwę portu w diagramie.
Bitstream: UCIO-1 / NSTD-1	Brak PACKAGE_PIN lub IOSTANDARD w XDC dla któregoś portu.
Licznik cały czas w 0 (Z1/Z3)	Przycisk działa odwrotnie (niewciśnięty = 1). Wstaw Utility Vector Logic: C_SIZE = 1, operacja **not**, między port Rst a Licznik Rst.
LD7 nie miga	ClockDivider nie dostaje zegara: clk_out1 → Clk_In. Port Clk_1Hz podłączony do Clk_Out?
Z3: licznik skacze o kilka kroków	CE podłączone do RotL/RotR bez OR, albo do licznika idzie inny zegar niż do RotaryEnc. Oba muszą mieć clk_out1.
Z3: w prawo liczy w dół	Podłącz RotL zamiast RotR do UPnDN.
Z3: enkoder nic nie robi	Płyta rozszerzająca wpięta? Linie ROT_* w XDC z LVCMOS33?
„No hardware target”	Kabel USB, zasilanie, ponownie Open target → Auto Connect.
Płyta działa po staremu	Po każdej zmianie: Generate Bitstream + Program device. Sprawdź, czy .bit jest z właściwego projektu (L2_Z1 / L2_Z3).

Złota zasada

Po zmianie: `Ctrl`+`S` → symulacja: **Relaunch** → płyta: **Generate Bitstream** → **Program device**.